FAT 12/16/32

The Basics

- FAT12/FAT16 (MS-DOS, Win95/98/NT/2000/XP)
  - Limited root directory size (512 entries)
  - No security features
  - 8.3 filenames
  - 12/16 bit cluster numbers
- FAT32 (Win95 w/ patches/98/2000/XP)
  - No limit on root directory size
  - No security features
  - Support for long filenames
- Basic difference is size of FAT entry—12, 16, or 32 bits.
  - Affects maximum number of addressable clusters
- Limited time/date information for FAT files
  - Last write date/time is always available
  - Creation date/time is optional and may not be available
  - Last access DATE ONLY is optional and may not be available

FAT Versions

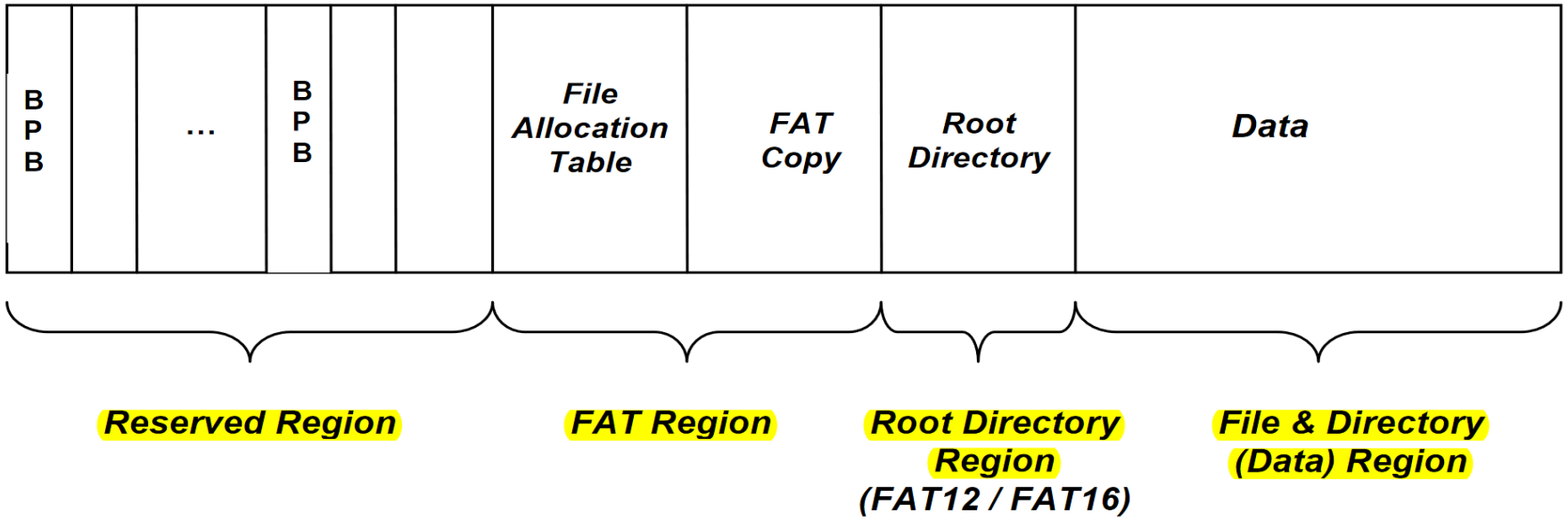
	FAT12	FAT16	FAT32
Max volume size	16 MB	2 GB / 4 GB	2 TB (practical)
Max file size	Limited by volume	2 GB	4 GB minus 1 byte
FAT entry width	12 bits (1.5 bytes)	16 bits (2 bytes)	32 bits (28 used)
Max clusters	4,084	65,524	~268 million
Cluster number range	0x002 - 0xFF6	0x0002 - 0xFFF6	0x0000002 - 0xFFFFFF6
OS support	DOS, all Windows	DOS, all Windows	Win95 SR2+, all modern
Use case	Floppy disks	Small drives, cards	USB, SD, cross-platform

The Details

- Complete reference:
  - ref/FAT32 Spec.pdf
- Notes
  - low-level details to perform investigations or data recovery
    - software suites handle low-level details
  - however
    - if FAT is destroyed or deliberately modified, however, some low-level hacking may be necessary
    - then, knowing details means the difference between success and failure
    - also, writing forensic tools often requires low-level details

The Details [2]

- All integers are little endian (LSB first)
- Four regions in FAT partition (in order)
  - Reserved Region
  - FAT Region
  - Root Directory Region (doesn't exist on FAT32 volumes)
  - File and Directory Data Region



Boot Sector / BPB

Field	Offset	Size	Description
BPB_BytsPerSec	11	2	Bytes per sector (typically 512)
BPB_SecPerClus	13	1	Sectors per cluster (power of 2)
BPB_RsvdSecCnt	14	2	Number of reserved sectors before first FAT
BPB_NumFATs	16	1	Number of FAT copies (usually 2)
BPB_RootEntCnt	17	2	Root directory entry count (0 for FAT32)
BPB_TotSec16	19	2	Total sector count, 16-bit (0 if > 65535)
BPB_FATSz16	22	2	Sectors per FAT (FAT12/16 only)
BPB_TotSec32	32	4	Total sector count, 32-bit
BPB_FATSz32	36	4	Sectors per FAT (FAT32 only)
BPB_RootClus	44	4	Root directory start cluster (FAT32 only)
BPB_FSInfo	48	2	FSInfo sector number (FAT32, usually 1)
BPB_BkBootSec	50	2	Backup boot sector location (FAT32, usually 6)

Calculating Key Offsets

- **FAT Region start:**
  - FATStart = BPB_RsvdSecCnt
- **Root Directory start (FAT12/16 only):**
  - RootDirStart = BPB_RsvdSecCnt + (BPB_NumFATs * BPB_FATSz16)
- **Root Directory size in sectors (FAT12/16):**
  - RootDirSectors = (BPB_RootEntCnt * 32 + BPB_BytsPerSec - 1) / BPB_BytsPerSec
- **Data Region start:**
  - DataStart = RootDirStart + RootDirSectors (FAT12/16)
  - DataStart = BPB_RsvdSecCnt + (BPB_NumFATs * BPB_FATSz32) for FAT32
- **Cluster to sector conversion:**
  - FirstSectorOfCluster(N) = DataStart + (N - 2) * BPB_SecPerClus
  - Note: clusters 0 and 1 are reserved; data starts at cluster 2
- This arithmetic is the core forensic skill for navigating raw images

FAT32 Differences: FSInfo and Backup Boot

- **FSInfo Structure (sector 1 by default)**
  - Provides hints for free space management
  - FSI_Free_Count: last known free cluster count
  - FSI_Nxt_Free: hint for where to start looking for free clusters
  - Forensic note: these are hints only, not authoritative
    - OS may not update them; compare against actual FAT scan
- **Backup Boot Sector (sector 6 by default)**
  - Exact copy of the boot sector at sector 0
  - Recovery strategy: if primary boot sector is corrupt, copy from sector 6
- **Root Directory is a cluster chain, not a fixed region**
  - BPB_RootClus (offset 44): starting cluster of root directory
  - Root directory can grow dynamically (no 512-entry limit)
  - Navigated the same way as any file: follow the FAT chain

FAT

- The FAT (File Allocation Table):

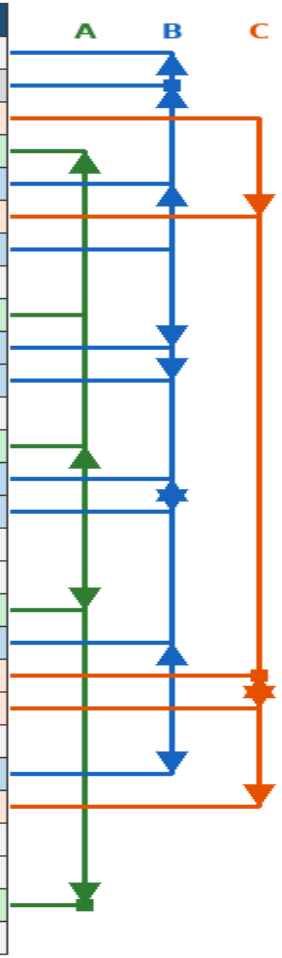
  an array one per cluster on the disk
- FAT12 is a little harder to deal with programmatically
  - » FAT entries are 1.5 bytes (12 bits) long
- FAT16/FAT32 are easier
  - » FAT entries are either 2 bytes or 4 bytes long
- FAT32
  - » most significant 4 bits in FAT entries are not used

User Directory

File	Start Clus
A	8
B	6
C	2

File Allocation Table

Idx	Value
0	22
1	Null
2	5
3	26
4	9
5	20
6	10
7	Free
8	17
9	1
10	14
11	Free
12	3
13	4
14	0
15	Free
16	Free
17	12
18	13
19	EOC
20	23
21	Free
22	18
23	19
24	Free
25	Free
26	Null
27	Free



File A File B File C Free EOC

Physical Blocks

Blk 0 B(4)	Blk 1 B(10)	Blk 2 C(1)	Blk 3 A(4)	Blk 4 B(8)	Blk 5 C(2)	Blk 6 B(1)
Blk 7 Free	Blk 8 A(1)	Blk 9 B(9)	Blk 10 B(2)	Blk 11 Free	Blk 12 A(3)	Blk 13 B(7)
Blk 14 B(3)	Blk 15 Free	Blk 16 Free	Blk 17 A(2)	Blk 18 B(6)	Blk 19 C(5)	Blk 20 C(3)
Blk 21 Free	Blk 22 B(5)	Blk 23 C(4)	Blk 24 Free	Blk 25 Free	Blk 26 A(5)	Blk 27 Free

FAT (Exercise: Trace the Chain

Using the color-coded FAT table on the right:

1. Trace File A

Start at cluster 8 (from directory)
Follow FAT[8], FAT[?], ... until EOC

2. Trace File B

Start at cluster 6

3. Trace File C

Start at cluster 2

How many clusters does each file occupy?

User Directory

File	Start Clus
A	8
B	6
C	2

File Allocation Table

Idx	Value
0	22
1	Null
2	5
3	26
4	9
5	20
6	10
7	Free
8	17
9	1
10	14
11	Free
12	3
13	4
14	0
15	Free
16	Free
17	12
18	13
19	EOC
20	23
21	Free
22	18
23	19
24	Free
25	Free
26	Null
27	Free

Exercise:
Trace each file chain through the FAT table.

File A | File B | File C | Free | EOC

Physical Blocks

Blk 0 B(4)	Blk 1 B(10)	Blk 2 C(1)	Blk 3 A(4)	Blk 4 B(8)	Blk 5 C(2)	Blk 6 B(1)
Blk 7 Free	Blk 8 A(1)	Blk 9 B(9)	Blk 10 B(2)	Blk 11 Free	Blk 12 A(3)	Blk 13 B(7)
Blk 14 B(3)	Blk 15 Free	Blk 16 Free	Blk 17 A(2)	Blk 18 B(6)	Blk 19 C(5)	Blk 20 C(3)
Blk 21 Free	Blk 22 B(5)	Blk 23 C(4)	Blk 24 Free	Blk 25 Free	Blk 26 A(5)	Blk 27 Free

Cluster Chain Walk-Through

- **Reading a file step by step:**
- 1. Read the directory entry to get the starting cluster number
  - Example: file REPORT.TXT has first cluster = 8
- 2. Go to the FAT and look up the entry at index 8
  - FAT[8] = 17 (next cluster is 17)
- 3. Read data from cluster 8, then look up FAT[17]
  - FAT[17] = 12 (next cluster is 12)
- 4. Continue following the chain:
  - FAT[12] = 3, FAT[3] = 26, FAT[26] = EOC marker
- 5. EOC (End Of Chain) marker means we have read all clusters
  - Full chain: 8 -> 17 -> 12 -> 3 -> 26 -> [end]
- Key insight: the FAT is essentially a linked list stored as an array
  - The index is the current cluster, the value is the next cluster
  - File data is scattered across the disk; the FAT connects the pieces

Special FAT Entry Values

- **Entry 0**: media type byte (copy of BPB_Media in low byte)
- **Entry 1**: end-of-chain marker (set by format utility)
  - May contain dirty volume / error flags on FAT16/32
- **Free cluster: 0x000 / 0x0000 / 0x00000000**
  - Cluster is available for allocation
- **Reserved: 0x001 / 0x0001 / 0x00000001**
  - Should not appear in a valid FAT
- **Bad cluster:**
  - FAT12: 0xFF7 | FAT16: 0xFFF7 | FAT32: 0x0FFFFFF7
  - Cluster has been marked as having physical defects
  - Forensic note: sometimes used to hide data (mark good clusters as bad)
- **End-of-chain (EOC):**
  - FAT12: 0xFF8-0xFFF | FAT16: 0xFFF8-0xFFFF | FAT32: 0x0FFFFFF8-0x0FFFFFFF
  - Any value in this range indicates last cluster of a file

Directory Entry Format

Offset	Size	Field	Description
0	11	DIR_Name	Short filename (8 + 3, space-padded)
11	1	DIR_Attr	File attributes byte
12	1	DIR_NTRes	Reserved for Windows NT (case flags)
13	1	DIR_CrtTimeTenth	Creation time, tenths of second (0-199)
14	2	DIR_CrtTime	Creation time (2-second granularity)
16	2	DIR_CrtDate	Creation date
18	2	DIR_LstAccDate	Last access date (date only, no time)
20	2	DIR_FstClusHI	First cluster number, high word (FAT32)
22	2	DIR_WrtTime	Last write time
24	2	DIR_WrtDate	Last write date
26	2	DIR_FstClusLO	First cluster number, low word
28	4	DIR_FileSize	File size in bytes (32-bit unsigned)

FAT: Dir Entry Filenames

- Short filenames (8 + 3 characters)
  - » If first byte of name is **0xE5** ➔ entry is deleted
  - » If first byte is **0x00** ➔ all entries following this one have first byte of name == 0 (free)
  - » If first byte is **0x05** ➔ first byte represents **0xE5** (valid Japanese char)
- These characters are illegal:
  - » Less than **0x20** (space), except **0x05** special case
  - » `0x22 -> (SYN)`
  - » `0x2A, 0x2B, 0x2C, 0x2E, 0x2F -> * + , - . /`
  - » `0x3A, 0x3B, 0x3C, 0x3D, 0x3E, 0x3F -> : ; < = > ?`
  - » `0x5B, 0x5C, 0x5D, 0x7C -> [ \ ] |`

Short Filename Storage

```
"foo.bar"        → "FOO     BAR"
"FOO.BAR"        → "FOO     BAR"
"Foo.Bar"        → "FOO     BAR"
"foo"            → "FOO        "
"foo."           → "FOO        "
"PICKLE.A"       → "PICKLE   A "
"prettybg.big"   → "PRETTYBGBIG"
```

- Notes:
  - case is **not** significant
  - "." between primary filename and extension is implied (not actually stored)
  - all entries are space-padded

Directory Entry: Attributes Byte

- Byte offset 11 in each 32-byte directory entry
- **Individual flag bits:**
  - 0x01 ATTR_READ_ONLY: file should not be modified
  - 0x02 ATTR_HIDDEN: file should not be shown in normal listings
  - 0x04 ATTR_SYSTEM: file is part of the operating system
  - 0x08 ATTR_VOLUME_ID: entry is the volume label (root directory only)
  - 0x10 ATTR_DIRECTORY: entry is a subdirectory, not a file
  - 0x20 ATTR_ARCHIVE: file has been modified since last backup
- **The long filename hack:**
  - ATTR_LONG_NAME = READ_ONLY | HIDDEN | SYSTEM | VOLUME_ID = 0x0F
  - Old FAT drivers see all four flags set and skip the entry entirely
  - New drivers recognize 0x0F as a long filename component
- Forensic note: hidden + system files are often worth examining
  - Malware frequently sets these flags to avoid casual discovery

Long filename support

- Long filename support is a hack:
  - » designed to be backwards compatible
  - » keeps to on-disk structures compatible with older FAT drivers, filesystem utilities, etc.
    - do not want **`chkdsk`** to try to "repair" long filename stuff!
- Long filenames are stored as a combination of
  - » a regular, short filename directory structure,
  - » followed by additional directory structures that will be ignored by older FAT versions

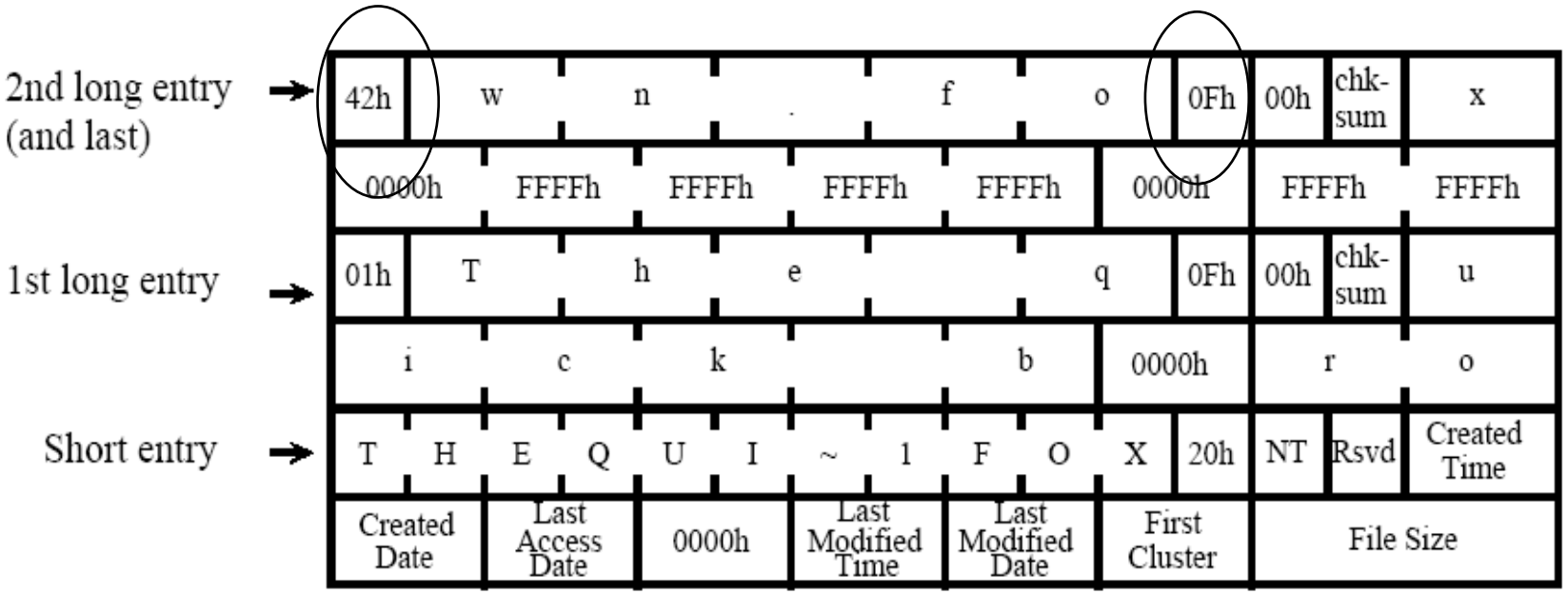
Long filenames rules

- Up to 255 characters in pathname component
- Total pathname no longer than 260 chars
- More supported characters
- Leading/trailing spaces ignored
- Internal spaces allowed
- Leading/embedded "." allowed
- Trailing "." are ignored

Long Filename Directory Entry

Offset	Size	Field	Description
0	1	LDIR_Ord	Sequence number (OR 0x40 for last entry)
1	10	LDIR_Name1	Characters 1-5 (Unicode, 2 bytes each)
11	1	LDIR_Attr	Attributes (always 0x0F)
12	1	LDIR_Type	Type (always 0x00 for VFAT)
13	1	LDIR_Chksum	Checksum of associated short name
14	12	LDIR_Name2	Characters 6-11 (Unicode, 2 bytes each)
26	2	LDIR_FstClusLO	First cluster (always 0x0000)
28	4	LDIR_Name3	Characters 12-13 (Unicode, 2 bytes each)

EXAMPLE: LONG & SHORT ENTRIES "the quick brown.fox"



◊ NB:

» short name does **not** contribute to long name

» long names preserve case & internal spaces

» however, **case is not significant** in matching filenames

FAT: TIME/DATE FORMAT

- Date format (by bit position):
  - 0–4: Day of month, valid value range 1-31 inclusive.
  - 5–8: Month of year, 1 = January, valid value range 1–12 inclusive.
  - 9–15: Count of years from 1980, valid value range 0–127 inclusive (1980–2107).
- Time Format:
  - A FAT directory entry time stamp is a 16-bit field that has a granularity of 2 seconds
  - 0–4: **2-second count**, valid value range 0–29 inclusive (0 – 58 seconds).
  - 5–10: **minutes**, valid value range 0–59 inclusive
  - 11–15: **hours**, valid value range 0–23 inclusive

Timestamp Precision and Coverage

- **Three timestamps available per directory entry:**
  - Write date/time: always populated, 2-second granularity
  - Creation date/time: optional, 2-second granularity + 10ms add-on
  - Access date: optional, date only (no time component)
- **Creation timestamp 10ms refinement:**
  - CrtTimeTenth (offset 13): 0-199 in units of 10ms
  - Adds 0-1.99 seconds to the creation time
  - Combined with the 2-second base, gives 10ms effective resolution
- **Which operations update which timestamps:**
  - File create: sets creation date/time and write date/time
  - File write/modify: updates write date/time only
  - File read/open: updates access date (if enabled)
  - Rename: typically does not update any timestamp
  - Copy: new creation date/time on copy; write date/time preserved

Timestamp Forensic Pitfalls

- **No timezone information stored**
  - FAT timestamps are assumed to be local time of the system
  - If a USB drive moves between time zones, timestamps become ambiguous
  - Daylight saving transitions can create duplicate or missing hours
- **Implementation inconsistencies**
  - Creation date and access date are optional per the spec
  - Windows populates all three; Linux often ignores creation time
  - Some embedded devices write no timestamps at all
- **The 2-second granularity problem**
  - Two files created 1 second apart may show identical timestamps
  - Sequence of operations cannot always be determined from timestamps alone
- **Anti-forensic timestamp manipulation**
  - Trivial to modify: just overwrite 4 bytes in the directory entry
  - Look for inconsistencies: access date before creation date, timestamps outside the 1980-2107 range, or timestamps that predate the OS installation
- Epoch: January 1, 1980. No dates before this can be represented

Deleted File Recovery

- **What happens when a file is deleted:**
  - First byte of the filename is overwritten with 0xE5
  - The cluster chain in the FAT is zeroed out (marked as free)
  - The file data itself is NOT erased from the data clusters
- **What remains intact after deletion:**
  - The rest of the directory entry (file size, starting cluster, timestamps)
  - The actual file content in the data region (until overwritten)
- **When recovery IS possible:**
  - File was contiguous (clusters in sequence): straightforward recovery
  - File was recently deleted and clusters have not been reallocated
- **When recovery is DIFFICULT or impossible:**
  - Fragmented file: FAT chain is gone, so cluster order is unknown
  - Clusters have been reallocated to new files
  - Volume has been defragmented since deletion
- **The 0xE5 marker is forensically significant:**
  - Its presence confirms a file existed and was actively deleted
  - Absence of 0xE5 (entry is 0x00) means slot was never used

Slack Space

- File slack: gap between logical end of file and end of last cluster
- **Two components of file slack:**
  - RAM slack (sector slack): bytes between file end and end of the last sector
    - Historically filled with whatever was in memory; modern OSes zero-fill
  - Drive slack: remaining sectors in the last cluster after the last file sector
    - Contains whatever was previously on those sectors
    - Old file content, fragments of deleted files
- **Example:**
  - Cluster size = 4096 bytes (8 sectors of 512 bytes)
  - File size = 1000 bytes
  - RAM slack = 512 - (1000 mod 512) = 24 bytes
  - Drive slack = 4096 - 1024 = 3072 bytes (6 remaining sectors)
- Volume slack: space between end of last partition and end of disk
- **Forensic significance:**
  - Slack space is a common location for residual evidence
  - Can contain fragments of previously deleted files
  - Also used deliberately for data hiding (anti-forensics)

Worked Example: Hex Dump Overview

```
Offset   00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0x0000   E5 45 50 4F 52 54 20 20  44 4F 43 20 00 64 63 8A  .EPORT  DOC .dc.
0x0010   25 59 25 59 00 00 63 8A  25 59 08 00 10 27 00 00  %Y%Y..c.%Y...'..
```

32-byte FAT directory entry — each field encodes file metadata.

The following slides decode each field, highlighting the relevant bytes.

Step 1: Filename (Bytes 0-10)

```
Offset    00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0x0000    E5 45 50 4F 52 54 20 20  44 4F 43 20 00 64 63 8A  .EPORT  DOC .dc.
0x0010    25 59 25 59 00 00 63 8A  25 59 08 00 10 27 00 00  %Y%Y..c.%Y...'..
```

Bytes 0-10: DIR_Name — Short filename in 8.3 format

Byte 0 = 0xE5: File has been DELETED

Bytes 1-10 = 45 50 4F 52 54 20 20 44 4F 43 = "EPORT DOC"

Reconstructed filename: R + EPORT.DOC = REPORT.DOC

(Replace 0xE5 with most likely original first character)

Step 2: Attributes (Byte 11)

```
Offset    00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0x0000    E5 45 50 4F 52 54 20 20  44 4F 43 20 00 64 63 8A  .EPORT  DOC .dc.
0x0010    25 59 25 59 00 00 63 8A  25 59 08 00 10 27 00 00  %Y%Y..c.%Y...'..
```

Byte 11: DIR_Attr — File attributes

Value = 0x20 = ATTR_ARCHIVE

Only the archive flag is set — this was a normal user file.
No hidden, system, or read-only flags. Not a directory or volume label.
Not 0x0F, so this is NOT a long filename entry.

Step 3: Timestamps (Bytes 13-19)

```
Offset   00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0x0000   E5 45 50 4F 52 54 20 20  44 4F 43 20 00 64 63 8A  .EPORT  DOC .dc.
0x0010   25 59 25 59 00 00 63 8A  25 59 08 00 10 27 00 00  %Y%Y..c.%Y...'..
```

Bytes 13-19: Timestamps — creation time/date, last access date

Byte 13 (CrtTimeTenth) = 0x64 = 100 = 1.00 second refinement

Bytes 14-15 (CrtTime) = 63 8A → 0x8A63 (LE) → 17:19:06

Decode: hours=(0x8A63>>11)=17, min=(0x8A63>>5)&0x3F=19, sec=(0x8A63&0x1F)*2=6

Bytes 16-17 (CrtDate) = 25 59 → 0x5925 (LE) → 2024-09-05

Decode: year=1980+(0x5925>>9)=2024, month=(0x5925>>5)&0xF=9, day=0x5925&0x1F=5

Bytes 18-19 (LstAccDate) = 25 59 → 2024-09-05 (date only, no time)

Step 4: Starting Cluster (Bytes 20-21, 26-27)

```
Offset   00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0x0000   E5 45 50 4F 52 54 20 20  44 4F 43 20 00 64 63 8A  .EPORT  DOC .dc.
0x0010   25 59 25 59 00 00 63 8A  25 59 08 00 10 27 00 00  %Y%Y..c.%Y...'..
```

Bytes 20-21, 26-27: DIR_FstClusHI + DIR_FstClusLO — Starting cluster

Bytes 20-21 (high word) = 00 00 → 0x0000
Bytes 26-27 (low word) = 08 00 → 0x0008 (LE)

Combined: (0x0000 << 16) | 0x0008 = cluster 8

This is where the file data begins in the data region.
(FAT chain is zeroed since file is deleted — assume contiguous.)

Step 5: File Size (Bytes 28-31)

```
Offset   00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0x0000   E5 45 50 4F 52 54 20 20  44 4F 43 20 00 64 63 8A  .EPORT  DOC .dc.
0x0010   25 59 25 59 00 00 63 8A  25 59 08 00 10 27 00 00  %Y%Y..c.%Y..'..
```

Bytes 28-31: DIR_FileSize — File size in bytes

Value = 10 27 00 00 → 0x00002710 (LE) = 10,000 bytes

With 4096-byte clusters: ceil(10000 / 4096) = 3 clusters needed.
Read clusters 8, 9, 10 (assuming contiguous), then truncate to 10,000 bytes.

Slack space: 3 * 4096 - 10000 = 2,288 bytes of potential residual data.

Step 6: Recovery Summary

```
Offset    00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F   ASCII
0x0000    E5 45 50 4F 52 54 20 20  44 4F 43 20 00 64 63 8A  .EPORT  DOC .dc.
0x0010    25 59 25 59 00 00 63 8A  25 59 08 00 10 27 00 00  %Y%Y..c.%Y...'..
```

Recovery Summary

1. File identified: REPORT.DOC (deleted — 0xE5 marker)
2. Created 2024-09-05 at 17:19:07 (including 1.00s refinement)
3. Starting cluster: 8 | File size: 10,000 bytes | Need 3 clusters
4. **Extract clusters 8-10 from data region, truncate to 10,000 bytes**
5. Examine 2,288 bytes of slack space for additional evidence